

Stock Price Trend Prediction with LSTM

A one-session-ahead forecasting study of Apple Inc.

TICKER	AAPL	FORECAST HORIZON	Next trading session
HISTORY	Five years of daily prices	REPORT DATE	30 September 2026

Executive Summary

This project builds a daily time-series pipeline that uses five years of split- and dividend-adjusted Apple (AAPL) market history to estimate the next trading session's adjusted closing price. It combines five market features, a 60-session lookback window, and a stacked Keras LSTM. The work includes data preparation, chronological model validation, a held-out test evaluation, saved model artifacts, plots, and a Streamlit dashboard.

On 181 held-out sessions from 9 January through 29 September 2026, the LSTM recorded a mean absolute error (MAE) of \$3.95 and a directional accuracy of 46.96%. A last-close baseline had a lower MAE of \$3.58, so the trained model did not outperform this simple reference on the saved test period. This is a reproducible forecasting exercise, not evidence of a reliable trading signal.

Evaluation takeaway The benchmark comparison is mixed: percentage error is 1.38%, but the LSTM's dollar MAE is about 10.3% higher than the last-close baseline, and directional accuracy is below 50%.

Measure	Result	Measure	Result
Usable observations	1,205	Input window	60 sessions
Train / validation / test	843 / 181 / 181	Epochs completed	13
Test MAE	\$3.95	Naive baseline MAE	\$3.58
Test RMSE / MAPE	\$5.30 / 1.38%	Directional accuracy	46.96%

1. Project Overview

Stock prices can react to information absent from historical price series. This project focuses on a bounded technical task: use recent daily market observations to estimate the next session's adjusted close, then compare that estimate with a chronological holdout and a simple baseline.

Objectives

- Retrieve daily AAPL price and volume history through yfinance and Yahoo Finance.
- Create moving-average, return, volume, and RSI features using information available up to each session.
- Train a Keras LSTM on sequential windows and preserve time order in train, validation, and test periods.
- Measure test error and direction accuracy, then compare the model with a last-close baseline.
- Package the notebook, model files, graphs, local dashboard, and static Vercel dashboard assets.

Workflow

01 Download Daily adjusted OHLCV	02 Engineer Returns, SMA gaps, RSI, volume	03 Sequence 60-session windows	04 Train Chronological LSTM fit	05 Evaluate Holdout, baseline, plots
--	--	--	---	--

Data and chronological split

The saved data covers approximately five years of daily AAPL history. The downloader requests auto-adjusted prices, so Close reflects split and dividend adjustments. Rolling indicators require an initial warm-up period; after that warm-up, 1,205 observations remain for modeling.

Subset	Rows	Share	Use
Training	843	70%	Fit model and scalers
Validation	181	15%	Monitor fit and callbacks
Test	181	15%	Final held-out evaluation

The held-out dates are 9 January 2026 to 29 September 2026. Samples are not shuffled. Feature and target scalers are fitted on training rows only, then applied to later periods.

2. Data Preparation and Technical Indicators

Each training example contains five values for each of the previous 60 sessions. The model predicts the next session's daily return. After prediction, the return is composed with the most recent observed close to produce an estimated adjusted close.

Model feature	Construction	Purpose
Daily_Return	$\text{Close}(t) / \text{Close}(t-1) - 1$	Short-term change; also the prediction target
SMA_20_Gap	$20\text{-day SMA} / \text{Close} - 1$	Position relative to the short moving average
SMA_50_Gap	$50\text{-day SMA} / \text{Close} - 1$	Position relative to the longer moving average
RSI_14	14-session Wilder-style RSI	Recent balance of upward and downward moves
Log_Volume	$\ln(1 + \text{daily volume})$	Scaled representation of trading activity

The chart displays adjusted close with 20- and 50-session simple moving averages and RSI(14). The model receives relative gaps from the moving averages rather than their raw price levels. Dashed RSI levels at 30 and 70 are visual reference levels, not model output thresholds.



Figure 1. AAPL adjusted close, moving averages, and RSI(14) across the saved history.

3. LSTM Model and Training

The network reads a sequence of 60 rows by five features and returns one scaled daily-return estimate. Two recurrent layers learn patterns across the lookback window; dropout and validation callbacks help constrain fitting on this small, single-ticker dataset.

Layer	Configuration	Output role
Input	60 time steps x 5 features	Recent market sequence
LSTM 1	32 units; return sequences	Sequence representation
Dropout	0.15	Regularization
LSTM 2	16 units	Compact sequence summary
Dropout	0.15	Regularization
Dense layers	16 ReLU, then 1 linear unit	Next-session return estimate

Training setup

Keras 3 uses the PyTorch backend. The optimizer is Adam with a 0.001 learning rate; mean squared error is the training loss. Training uses batches of 64, preserves sequence order, and is seeded with 42. The run was configured for up to 20 epochs. Early stopping monitored validation loss with patience 8 and restored the best weights; a second callback halves the learning rate after four stagnant validation epochs. Training ended after 13 epochs.

Price reconstruction The model predicts return $\hat{r}$. The price estimate is previous adjusted close $\times (1 + \hat{r})$. Each forecast is anchored to the latest observed close.

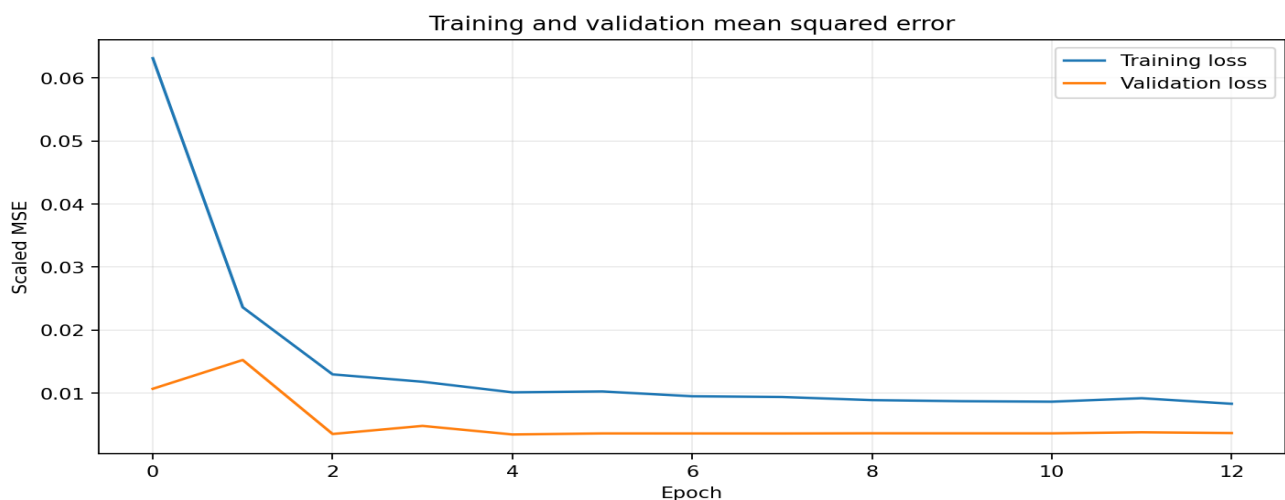


Figure 2. Training and validation loss in scaled target space. This curve is a fit diagnostic, not the dollar-price test metric.

4. Test Results

The final comparison uses 181 one-step forecasts over the chronological test period. For every test date, the model uses the preceding 60 observed sessions. This is a rolling one-step test, not a recursive forecast over an unobserved future path.

Metric	LSTM result	Interpretation
MAE	\$3.95	Average absolute price error in USD
RMSE	\$5.30	Penalizes larger errors more heavily
MAPE	1.38%	Average absolute error as a share of actual price
Directional accuracy	46.96%	Correct sign of the one-day move vs previous close
Last-close baseline MAE	\$3.58	Baseline predicts no change from previous close

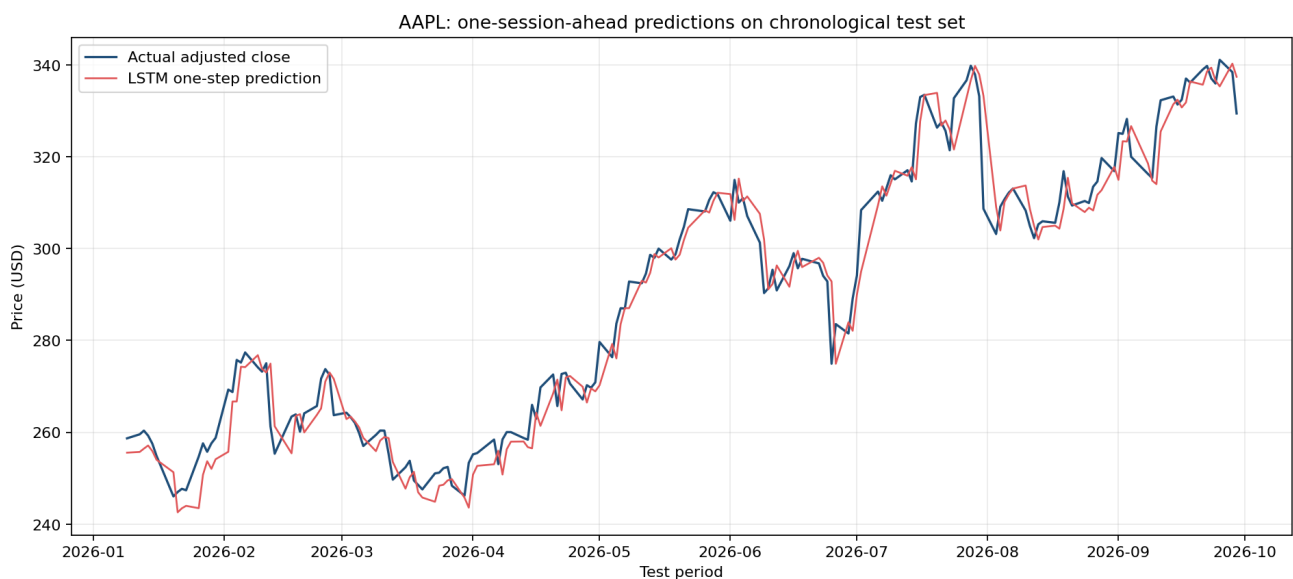


Figure 3. Actual adjusted close and LSTM one-step estimates on the 2026 held-out test period.

What the results show The predicted series follows several broad movements, but its MAE is about 10.3% higher than the last-close baseline. Directional accuracy is below 50%. On the saved results, the LSTM does not demonstrate improvement over the simple baseline.

5. Latest Forecast and Interpretation

AS OF 2026-09-29	LAST ADJUSTED CLOSE \$329.40	NEXT-SESSION ESTIMATE \$328.27
---------------------	---------------------------------	-----------------------------------

The saved point estimate implies a -0.34% change from the 29 September 2026 adjusted close. It is a one-session forecast generated from the latest 60 observed feature rows. No realized next-session result is stored with the project artifacts, so this report does not claim the estimate matched a later price.

Limitations

- The experiment covers one company and one historical test window; results may change across tickers and market conditions.
- Inputs are historical price, volume, and derived indicators. Earnings, news, macroeconomic variables, and wider market context are not included.
- The baseline has lower MAE than the LSTM on this test. A more complex model is not automatically a better forecast.
- The forecast is a point estimate with no uncertainty interval. Evaluation does not include transaction costs or a simulated trading strategy.
- Yahoo Finance access is through the community-maintained yfinance package; data availability and adjustments can change.

Conclusion and next steps

The project delivers a reproducible end-to-end LSTM workflow with technical indicators, chronological validation, saved weights, evaluation graphs, and a dashboard. The test comparison shows why a baseline matters: the model did not beat a last-close prediction on this holdout. A stronger next study would use walk-forward evaluation across several periods and tickers, compare additional simple and statistical baselines, and report forecast uncertainty.

Use This is an educational time-series example, not financial advice or a standalone basis for investment decisions.

6. Deliverables and Reproducibility

The project folder contains runnable source, an executed notebook, trained model files, saved data, evaluation results, graphs, and dashboard assets. The GitHub repository is the shareable source location; the local path is the Windows working copy used for this report.

Deliverable	What it contains
Stock_Price_Trend_Prediction_LSTM.ipynb	Walkthrough of data, features, model, evaluation, and plots
AAPL_lstm.weights.h5 + AAPL_scalers.joblib	Trained weights and fitted feature/target scalers
AAPL_history.csv + AAPL_raw_history.csv	Feature-ready and raw adjusted history
metrics.json + forecast.json	Machine-readable test results and latest point estimate
Three PNG graphs	Indicators, test predictions, and training history
streamlit_app.py, stock_lstm.py, train.py	Dashboard, forecasting pipeline, and training entry point
vercel_site/	Static dashboard assets for Vercel deployment

Run the local dashboard

On Windows, double-click launch_app.bat. It creates a .venv environment, installs packages from requirements.txt, then starts Streamlit. Python 3.10 or newer is required. To retrain from a terminal in the project folder, run:

```
python train.py --ticker AAPL --period 5y
```

The local Streamlit dashboard runs at <http://localhost:8501>. Vercel serves the static dashboard from vercel_site; Keras inference and interactive Streamlit refresh run locally.

Project locations

GitHub: github.com/jitendradurgaprasad/stock_price_prediction

C:\Users\M jitendra\Documents\Codex\2026-09-30\3-stock-price-trend-prediction-with\outputs

References

Project implementation and saved run: stock_lstm.py, train.py, metrics.json, forecast.json, and the executed notebook in the project folder.

yfinance documentation: ranaroussi.github.io/yfinance

Keras LSTM layer API: keras.io/api/layers/recurrent_layers/lstm

scikit-learn MinMaxScaler API: scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler